

Final Project: Attack & Defend Your Own System

1. Goal

In the midterm you built an LLM system. In the final you will break it, then defend it — the two halves of the security mindset this course teaches.

Each group picks one attack/defense pair from Section 5, reproduces both against the system they built, and reports honestly on what the defense stops and what still gets through.

The core principle: **a security claim must state what still gets through, not only what is stopped**. A project reporting “the defense cut attack success from 90% to 12%” and then explaining the residual 12% scores higher than one reporting “the defense works.”

2. What each group does

1. Assign one attack/defense pair from Section 5.
2. Reproduce the **attack** against your own midterm system and measure how well it works.
3. Reproduce the **defense** and measure how much it reduces the attack — and what it costs in utility, latency, and false positives.
4. Analyze: what the defense stops, what still gets through, and what access each result assumes.

3. What “done” looks like

Before writing any code, your group should be able to state:

- The **attack metric** you will report (e.g., attack success rate — the fraction of targeted queries that produce the attacker’s chosen outcome).
- The **utility metric** the defense must not destroy (e.g., task accuracy / answer quality on benign inputs).
- The **cost axes** you will measure (added latency, compute, false-refusal rate on benign inputs).
- A **baseline**: attack success on your undefended midterm system.

A finished project reports all four, for the attack alone and for attack-plus-defense, with at least one honest sentence on what the defense does not fix.

4. Deliverables & timeline

Deliverable	Contents	Weight
Proposal (1 page)	Chosen pair, your system, metrics, baseline plan	10%
Code + README	Attack script, defense script, evaluation harness, run instructions	35%
Live demo	Attack succeeds on undefended system; defense measurably reduces it	20%
Written report (6–8 pp)	Threat model, method, results table, residual-gap analysis, limitations	20%

Deliverable	Contents	Weight
Presentation + Q&A	Present results; answer “what’s your weakest assumption?”	15%

Suggested rhythm: proposal by week 1, working attack by week 3, working defense by week 5, demo and report in the final week. Two to four students per group.

5. Attack/Defense pair

If your system is...	Best-fit pair(s)	Why
A RAG chatbot with a knowledge base	Pair 3 (RAG poisoning); also Pair 1	The knowledge base is the attack surface
An LLM agent with tools / external data	Pair 1 (prompt injection)	Untrusted tool/web output carries injected instructions
A guardrailed chatbot on an aligned model	Pair 2 (jailbreak)	Directly targets the refusal behavior
A system you fine-tuned / via a FT API	Pair 5 (FT strips alignment); Pair 4	Fine-tuning is the injection vector
Built on a third-party fine-tuned model	Pair 4 (backdoor)	You cannot rule out a pre-planted trigger

6. The five attack/defense pairs

Each pair lists the attack, the defense, what to demonstrate, and the main pitfall. Every repository is public.

Pair 1 — Prompt Injection

Maps to Lesson 2

Attack — Open-Prompt-Injection. Liu, Jia, Geng, Jia & Gong.

Paper: USENIX Security 2024 **Code:** github.com/liu00222/Open-Prompt-Injection

A benchmark implementing several injection attacks (naive, escape-character, context-ignoring, fake-completion, combined) and several defenses — one repo covers both halves.

Defense — StruQ (Structured Queries). Chen, Piet, Sitawarin & Wagner.

Paper: USENIX Security 2025 **Code:** github.com/Sizhe-Chen/StruQ

Separates the trusted-prompt channel from the untrusted-data channel and fine-tunes the model to obey only the prompt channel. Reports under 2% attack success on optimization-free attacks with little utility loss.

Alternative defense: SecAlign (Chen et al., CCS 2025), github.com/facebookresearch/SecAlign — a preference-optimization defense.

What to demonstrate: Inject an instruction into a data field your system processes (a retrieved doc, a tool result, an uploaded file). Show the undefended system obeys it; show StruQ reduces its success. Report attack success before/after and benign-task utility.

Watch out for: *StruQ reduces but does not eliminate optimization-based injections — probe exactly there in your residual-gap analysis.*

Pair 2 — Optimization-Based Jailbreak

Maps to Lesson 2

Attack — GCG (Greedy Coordinate Gradient). Zou, Wang, Carlini, et al.

Paper: arXiv 2023 (widely used; not a main-track paper) **Code:** github.com/llm-attacks/llm-attacks

Optimizes an adversarial suffix that coerces an aligned model into an affirmative harmful response.

Defense — SmoothLLM. Robey, Wong, Hassani & Pappas.

Paper: arXiv 2023 (widely used; not a main-track paper) **Code:** github.com/arobey1/smooth-llm

Randomly perturbs multiple copies of the input and aggregates the responses, exploiting the brittleness of GCG suffixes to character-level noise. Reduces GCG attack success to about 1% on the models tested.

What to demonstrate: Run GCG against your system's aligned base model to elicit a refused behavior; wrap the same model in SmoothLLM and show the attack success rate collapse. Report attack success before/after, SmoothLLM's added query cost, and any utility drop on benign prompts.

Watch out for: *Neither GCG nor SmoothLLM is a main-track conference paper — both are influential arXiv works with strong code. SmoothLLM is also weak against semantically coherent jailbreaks (AutoDAN, persuasive attacks); demonstrating that limit strengthens your report.*

Pair 3 — RAG Knowledge-Base Poisoning

Maps to Lesson 3

Attack — PoisonedRAG. Zou, Geng, Wang & Jia.

Paper: USENIX Security 2025 **Code:** github.com/sleepeer/PoisonedRAG

Injects a handful of crafted documents into the knowledge base so the retriever surfaces them and the LLM emits an attacker-chosen answer. As few as ~5 malicious texts per target question reach ~90% attack success.

Defense — RAGDefender. Kim, Lee & Koo.

Paper: ACSAC 2025 **Code:** github.com/SecAI-Lab/RAGDefender

Lightweight ML-based detection of adversarial passages before generation. MIT-licensed and ships with PoisonedRAG evaluation scripts.

Alternative defense: RobustRAG (Xiang et al.) — the classic “isolate-then-aggregate” defense: answer from each retrieved passage in isolation, then aggregate so a few poisoned passages can’t dominate. Confirm the authors’ official repo before use.

What to demonstrate: Poison your RAG system’s knowledge base for a chosen target question; show the answer flips to the attacker’s target; apply RAGDefender and show accuracy recover / attack success drop. Report attack success and clean accuracy before/after, plus the defense’s runtime overhead.

Watch out for: *Current RAG defenses leave residual risk on technical or narrow-domain corpora — if your knowledge base is domain-specific, test there.*

Pair 4 — Backdoor Attack

Maps to Lesson 3

Attack — BackdoorLLM. Li, Huang, Zhao, Ma & Sun.

Paper: NeurIPS 2025 (Datasets & Benchmarks Track) **Code:** github.com/bboylyg/BackdoorLLM

A benchmark implementing multiple backdoor families (data-poisoning triggers, weight-poisoning, chain-of-thought backdoors). Pick one trigger and behavior for your demo. Use the paper authors’ official repository.

Defense — CROW (Internal Consistency Regularization). Min, Pham, Li & Sun.

Paper: ICML 2025 **Code:** github.com/NayMyatMin/CROW

Neutralizes backdoors by enforcing smooth layer-wise hidden-state transitions during a short clean-data fine-tune; needs no trigger knowledge and no clean reference model.

Alternative defense: BEEAR (Zeng et al.), github.com/reds-lab/BEEAR — embedding-based removal of safety backdoors, effective in minutes.

What to demonstrate: If your system used a fine-tuned model, plant a backdoor with a chosen trigger (benign inputs behave normally; the trigger flips behavior); then run CROW and show the trigger stops firing while benign accuracy holds. Report backdoor success rate before/after and clean accuracy.

Watch out for: *This pair requires that you can fine-tune your model — feasible on a 7B model with LoRA. API-only groups should pick a different pair.*

Pair 5 — Fine-Tuning Strips Safety Alignment

Maps to Lesson 4

Attack — Attack via Overfitting (“TenBenign”). Xie, Song & Luo.

Paper: NeurIPS 2025 **Code:** github.com/ZHIXINXIE/ten_benign

Jailbreaks an aligned model by fine-tuning on only ten benign QA pairs: first overfit to a single refusal answer, then fine-tune on normal answers so the model “forgets” to refuse. Because the data is benign, it evades moderation filters.

Defense — Booster. Huang, Hu, Ilhan, Tekin & Liu.

Paper: ICLR 2025 (Oral) **Code:** github.com/git-disl/Booster

An alignment-stage defense: simulate the harmful perturbation during alignment and train the model to resist it, so later fine-tuning does less damage.

Alternative defense: Vaccine (Huang et al., NeurIPS 2024), same group’s repo family — perturbation-aware alignment that keeps hidden embeddings invariant under harmful fine-tuning.

What to demonstrate: Take your system’s aligned model, run the TenBenign attack, and show refusal rate collapse; then re-align the base with Booster before the same attack and show it blunted. Report harmful-response rate before/after, undefended vs. Booster-aligned, plus benign task performance.

Watch out for: *This measures robustness to one fine-tuning attack; note that alignment-stage defenses trade some benign performance and are not a guarantee against stronger or adaptive attacks.*

7. Grading rubric

Axis	Strong	Weak
Reproduction	Attack and defense both run on your own system	Only the repo’s canned demo runs
Measurement	Clear attack success + utility + cost, with a baseline	Single number, no baseline
Residual-gap analysis	Names what still gets through and why	“The defense works”
Threat-model honesty	States the access each result assumes	Implies universal effectiveness
Presentation Q&A	Answers “what’s your weakest assumption?”	Cannot identify own limitations

The highest-value skill assessed is the residual-gap analysis. Every defense here has documented failure modes — finding and demonstrating your pair’s failure mode is the point of the project, not a mark against you.